DevEval: A Manually-Annotated Code Generation Benchmark Aligned with Real-World Code Repositories

Jia Li ♂[1,2], Ge Li[1,2], Yunfei Zhao[1,2], Yongmin Li[1,2], Huanyu Liu[1,2], Hao Zhu[1,2], Lecheng Wang[1,2]
Kaibo Liu[1,2], Zheng Fang[1,2], Lanshen Wang[1,2], Jiazheng Ding[1,2], Xuanming Zhang[1,2]
Yuqi Zhu[1,2], Yihong Dong[1,2], Zhi Jin[1,2], Binhua Li[3], Fei Huang[3], Yongbin Li[3]
[1]School of Computer Science, Peking University
[2] Key Laboratory of High Confidence Software Technologies (Peking University), Ministry of Education
[3]Alibaba Group
lijia@stu.pku.edu.cn, {lige, zhijin}@pku.edu.cn

arXiv:2405.19856v1 [cs.CL] 30 May 2024

Abstract

How to evaluate the coding abilities of Large Language Models (LLMs) remains an open question. We find that existing benchmarks are poorly aligned with real-world code repositories and are insufficient to evaluate the coding abilities of LLMs.

To address the knowledge gap, we propose a new benchmark named **DevEval**, which has three advances. ❶ DevEval aligns with real-world repositories in multiple dimensions, *e.g.,* code distributions and dependency distributions. ❷ DevEval is annotated by 13 developers and contains comprehensive annotations (*e.g.,* requirements, original repositories, reference code, and reference dependencies). ❸ DevEval comprises 1,874 testing samples from 117 repositories, covering 10 popular domains (*e.g.,* Internet, Database). Based on DevEval, we propose **repository-level code generation** and evaluate 8 popular LLMs on DevEval (*e.g.,* gpt-4, gpt-3.5, StarCoder 2, DeepSeek Coder, CodeLLaMa). Our experiments reveal these LLMs' coding abilities in real-world code repositories. **For example, the highest Pass@1 of gpt-4-turbo only is 53.04% in our experiments.** We also analyze LLMs' failed cases and summarize their shortcomings. We hope DevEval can facilitate the development of LLMs in real code repositories. DevEval, prompts, and LLMs' predictions have been released[1].

1 Introduction

Code generation with Large Language Models (LLMs) has attracted lots of researchers' attention (Guo et al., 2024; Rozière et al., 2023; Lozhkov et al., 2024), and some commercial products have been produced, *e.g.,* GitHub Copilot (GitHub, 2023). With more and more LLMs emerging, how to evaluate LLMs on code generation remains an open question.

[1]https://github.com/seketeam/DevEval

```
def has_close_elements(numbers, threshold):
  for idx, elem in enumerate(numbers):
    for idx2, elem2 in enumerate(numbers):
      if idx != idx2:
        distance = abs(elem - elem2)
        if distance < threshold:
          return True
  return False
```

(a) A standalone function in HumanEval

```
# imapclient.IMAPClient.namespace
def namespace(self):
  data = self._command_and_check("namespace")
  parts = []
  for item in parse_response(data):
    (more lines . . )
      for prefix, separator in item:
        if self.folder_encode:
          prefix = decode_utf7(prefix)
        converted.append((prefix, to_unicode)
      parts.append(tuple(converted))
  return Namespace(*parts)
```

(b) A non-standalone function in a real-world project

Figure 1: Examples of standalone and non-standalone functions. Dependencies are highlighted, *i.e.,* yellow: intra-class dependencies, green: intra-file dependencies, and blue: cross-file dependencies.

Existing benchmarks are mainly composed of hand-crafted programming problems and are poorly aligned with real-world code repositories. LLMs' performance on these benchmarks is inconsistent with developers' actual experiences in real-world software development. Thus, a benchmark aligned with real-world repositories is necessary. We analyze over 1 million functions from 500 real-world repositories (see Section 3) and think a good benchmark should satisfy the following features.

- **Real-world Repository.** The benchmark should be collected from real-world code repositories (Yu et al., 2023).
- **Real Code Distribution.** Real-world repositories comprise two types of code, *i.e.,* standalone and non-standalone code. As shown in Figure 1, a standalone function solely uses built-in or public libraries, while a non-standalone one contains

Table 1: The comparison between existing benchmarks and DevEval.

Benchmark	Real Repo.	Real Code Distribution	Comprehensive Annota.	Robust Metric
CoNaLA (Yin et al., 2018)	✖	✖	✖	✖
Concode (Iyer et al., 2018)	✔	✖	✖	✖
HumanEval (Chen et al., 2021)	✖	✖	✖	✖
MBPP (Austin et al., 2021)	✖	✖	✖	✖
APPS (Hendrycks et al., 2021)	✖	✖	✖	✖
PandasEval (Zan et al., 2022)	✖	✖	✖	✖
NumpyEval (Zan et al., 2022)	✖	✖	✖	✖
AixBench (Li et al., 2023b)	✔	✖	✖	✖
ClassEval (Du et al., 2023)	✖	✖	✖	✖
CoderEval (Yu et al., 2023)	✔	✖	✖	✔
DevEval (Ours)	✔	✔	✔	✔

context-aware *dependencies* (*i.e.,* invocations of code elements defined in current repositories). The benchmark should cover both types of code and ensure their ratios are realistic. The number of dependencies should also be consistent with real-world repositories.

- **Comprehensive Annotations.** The benchmark can offer comprehensive annotations, including natural language requirements, original repositories, and ground truths (code and dependencies).

- **Robust Evaluation Metrics.** The benchmark should provide execution-based metrics (*e.g.,* Pass@k) evaluate functional correctness of programs and metrics to assess the accuracy of dependencies in programs.

However, as shown in Table 1, none of the existing benchmarks satisfies all aforementioned features. The problem hinders the evaluation and development of LLMs in the real development process.

To address the above problem, we propose a new code generation benchmark named DevEval, which aligns with real-world code repositories. As shown in Table 1, DevEval satisfies the above features. ❶ DevEval comprises 1,874 testing samples from 117 real-world repositories, which cover 10 popular domains (*e.g.,* Internet, Database). ❷ DevEval is constructed through a rigorous pipeline and aligns with real-world repositories. Specifically, the distributions of code and dependencies in DevEval are consistent with the ones in 500 real-world repositories. Detailed statistics are in Section 2.4. ❸ DevEval is annotated by 13 developers and contains comprehensive annotations, *e.g.,* detailed requirements, original repositories, reference code, and reference dependencies. ❹ DevEval leverages test cases to check models' predictions and report Pass@k. It also proposes Recall@k to evaluate the dependencies in predictions.

Based on DevEval, we propose **repository-level code generation**, which simulates the developers' coding process in a working repository. The task asks models to write the code based on requirements and a complete repository.

We evaluate 8 popular LLMs (*i.e.,* gpt-4 (OpenAI, 2023b), gpt-3.5 (OpenAI, 2023a), DeepSeek Coder (Guo et al., 2024), StarCoder 2 (Lozhkov et al., 2024), CodeLLaMa (Rozière et al., 2023)). These LLMs exhibit low performance on DevEval, especially compared to their performance on previous benchmarks. **For example, gpt-4-turbo-1106 achieves a Pass@1 score of 80% on HumanEval, while its highest Pass@1 on DevEval is only 53.04%.** Our results reveal the coding abilities of these LLMs in real-world repositories. We further analyze failed cases and summarize the shortcomings of existing LLMs in DevEval.

In summary, our contributions are as follows:

- We summarize four features (see Table 1) that a code generation benchmark for real-world repositories should satisfy.

- We propose a new code generation benchmark - DevEval, satisfying the above features. The benchmark has been released.

- We propose repository-level code generation, which provides a challenging and realistic evaluation scenario.

- We evaluate 8 popular LLMs on DevEval, analyzing their strengths and shortcomings in repository-level code generation.

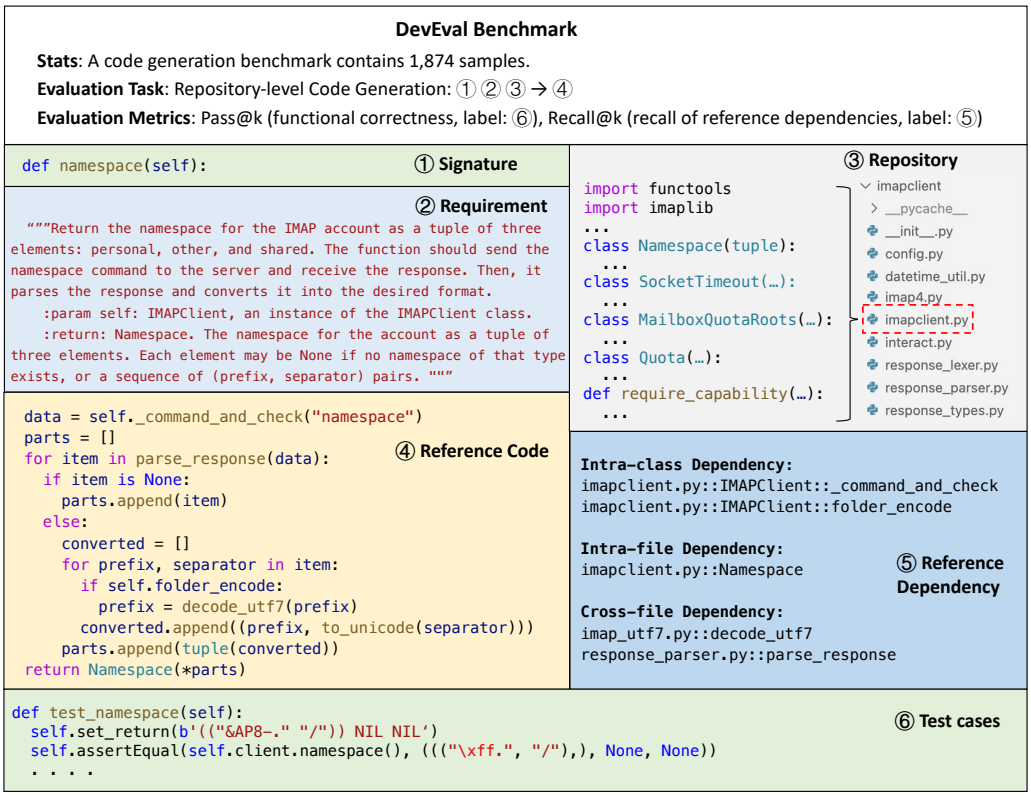

Figure 2: An overview of DevEval. Each sample consists of six components.

We hope DevEval can align with the actual experiences of developers during the practical development process. By DevEval, practitioners can pick up superior LLMs and facilitate the application of code generation techniques in real-world repositories.

2 Benchmark - DevEval

2.1 Overview

DevEval contains 1,874 samples derived from 117 real-world code repositories. As shown in Figure 2, each sample consists of six components. ❶ **Function Signature:** The signature of the target code. ❷ **Requirement:** An English description detailing the functionality of the target code. ❸ **Repository Contexts:** Code contexts (*e.g.,* classes, functions, variables) defined outside the target code in the current repository. ❹ **Reference Code:** A developer-written implementation of the target code. This code may invoke dependencies defined in the current repository. ❺ **Reference Dependency:** The dependencies invoked in the reference code include intra-class, intra-file, and cross-file dependencies. ❻ **Test Cases:** Test cases are used to check the functional correctness of the code.

2.2 Task Definition

Based on DevEval, we propose **repository-level code generation** task. A model is given a function signature, a requirement, and a complete repository. The model is asked to output a function to satisfy the requirement. We then insert the function into its repository and check its correctness.

2.3 Evaluation Metrics

Pass@k (Functional Correctness). Following previous studies (Chen et al., 2021; Austin et al., 2021; Yu et al., 2023), we assess the functional correctness of programs by executing test cases and compute the unbiased Pass@k. Specifically, we generate $n \geq k$ programs per requirement, count the number of correct programs $c \leq n$ that pass test cases, and calculate the Pass@k:

$$\text{Pass@}k := \mathop{\mathbb{E}}_{\text{Requirements}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right] \quad (1)$$

Recall@k (Recall of Reference Dependency). Besides the functional correctness, we expect LLMs to invoke relevant dependencies defined in contexts. Hence, we propose Recall@k, which

Table 2: The comparison between popular code generation benchmarks and DevEval. SA: Standalone. L(Re): the average lengths (tokens) of requirements.

Benchmark	Code Distribution				Dependency				Repository's Scale		#L(Re)
	#Repo	#Total	SA (%)	Non-SA (%)	#Type	#Total	#Per Sample	Path	#File	#Line	
CoNaLA (Yin et al., 2018)	–	500	100%	0%	0	0	0	✗	0	0	13.1
HumanEval (Chen et al., 2021)	–	164	100%	0%	0	0	0	✗	0	0	58.8
MBPP (Austin et al., 2021)	–	974	100%	0%	0	0	0	✗	0	0	16.1
PandasEval (Zan et al., 2022)	–	101	100%	0%	0	0	0	✗	0	0	29.7
NumpyEval (Zan et al., 2022)	–	101	100%	0%	0	0	0	✗	0	0	30.5
AixBench (Li et al., 2023b)	–	175	100%	0%	0	0	0	✗	0	0	34.5
ClassEval (Du et al., 2023)	–	100	100%	0%	0	0	0	✗	0	0	–
Concode (Iyer et al., 2018)	–	2,000	20%	80%	1	2,455	1.23	✗	0	0	16.8
CoderEval (Yu et al., 2023)	43	230	36%	64%	3	256	1.73	✗	71	14,572	41.5
DevEval	**117**	**1,874**	**27%**	**73%**	**3**	**4,672**	**3.41**	✓	**243**	**45,941**	**91.5**
500 Real-world Repos	500	1M	27%	73%	3	3M	3.22	–	238	46,313	–

Table 3: The distribution of dependency types. The values in parentheses are the corresponding percentages in all dependencies.

Dependency Type	HumanEval	Concode	CoderEval	DevEval	500 Projects
Intra-class	0	2,455 (100%)	117 (46%)	**1,778 (38%)**	939k (42%)
Intra-file	0	0	90 (35%)	**1,502 (32%)**	597k (29%)
Cross-file	0	0	49 (19%)	**1,392 (30%)**	611k (30%)

gauges the recall of reference dependencies in generated programs.

Specifically, LLMs generate k programs per requirement. For the i-th program, we employ a parser[2] to extract its dependencies as $\mathbb{P}_i$. Subsequently, we compare $\mathbb{P}_i$ with reference dependencies $\mathbb{R}$ and compute the Recall@k:

$$\text{Recall@}k := \mathop{\mathbb{E}}_{\text{Requirements}} \left[\max_{i \in [1,k]} \frac{|\mathbb{R} \cap \mathbb{P}_i|}{|\mathbb{R}|} \right] \quad (2)$$

where $|\cdot|$ means the number of elements of a set.

2.4 Features of DevEval

Compared to existing benchmarks, DevEval shows three unique advances, which we discuss below.

❶ Alignment with real-world code repositories. Table 2 shows the data distributions of existing benchmarks and DevEval. We also show the data distributions of 500 real-world repositories and consider them as the oracle. We can see that DevEval aligns with 500 real repositories in multiple aspects, *i.e.,* code distributions, the number of dependencies, and the scale of repositories. Table 3 further shows the distribution of dependency types, *i.e.,* intra-class, intra-file, and cross-file dependencies. DevEval outperforms previous benchmarks in all types, showing a distribution that is close to the distribution in 500 real repositories.

[2]We develop the parser based on an open-source static analysis tool - Pyan (Pyan, 2023).

❷ Comprehensive Annotations. As shown in Figure 2, DevEval provides comprehensive annotations that are labeled by 13 human developers. Particularly, DevEval has advantages in requirements and reference dependencies.

Requirements. Original code comments in repositories often are vague and are different from requirements in practice. We engaged 13 developers to write requirements, costing approximately 674 person-hours manually. As depicted in Figure 2, each requirement encapsulates the code's functionality and input-output parameters. The average length of requirements in DevEval (91.5 tokens) more than doubles that of CoderEval (41.5 tokens). *Reference Dependencies.* Previous benchmarks (*i.e.,* CoderEval, ClassEval) only provide dependencies' names (*e.g.,* `close`). Because many functions have the same name in practice, it is hard to identify whether generated dependencies are correct by relying on names. DevEval annotates dependencies with paths (*e.g.,* `A.py::ClassB::close`), addressing ambiguity and biases. These annotations offer a broad arena to explore repository-level code generation and evaluation.

❸ A realistic task and evaluation metrics. Traditional benchmarks fall into a simple requirement-to-code task. In contrast, DevEval proposes a more realistic task - repository-level code generation. This task simulates the coding process of developers in a working repository. Besides, we design two metrics to comprehensively assess the correctness of generated programs in functionality and dependencies.

❹ Wide scope for research communities. The DevEval and repository-level code generation can serve as an arena to compare approaches ranging from retrieval and long-context models to decision-

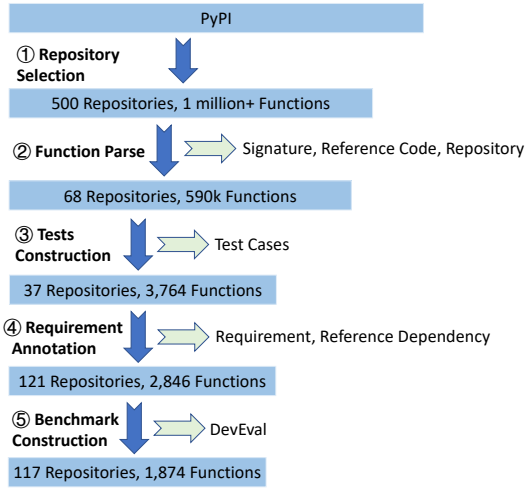

Figure 3: The process of collecting DevEval.

making agents. DevEval also allows creative freedom, as models can generate diverse programs to meet requirements.

3 Benchmark Construction

As shown in Figure 3, the collection of DevEval consists of five stages.

Stage ❶: Repository Selection. PyPI (PyPI) is a rich data source for real code repositories. We identify the top 10 popular programming topics in PyPI and select the top 50 repositories under each topic. The selection follows three criteria: open-source licenses, non-fork and non-malicious repositories, and explicit unit tests. We download the latest released versions in November 2023 and finally obtain 500 practical projects (10 topics * 50 projects).

Stage ❷: Function Parse. We extract functions from 500 repositories and exclude trivial functions (*i.e.,* empty or initialization functions). We extract each function's signature and function body (*i.e.,* reference code). The other programs with current repositories are considered as *repository contexts*. Finally, this stage obtain 590,365 candidate functions.

Stage ❸: Tests Construction. We extract test cases from repositories invoking specific candidate functions. We use a public framework - `setuptools` to automatically install running environments and run test cases with a popular testing framework - `Pytest`. Candidate functions without executable test cases are excluded. Meanwhile, we ensure our test cases succeed in the reference code and fail in the wrong programs. Finally, we retain 3,764 candidate functions.

Table 4: Studied LLMs in this paper. Context L.: Context Window.

Type	Name	Version	Context W.
Closed-source	gpt-4	gpt-4-turbo-1106	128,000
	gpt-3.5	gpt-3.5-turbo-1106	16,385
Open-source	StarCoder 2	15B	16,384
	StarCoder 2	7B	16,384
	DeepSeek Coder	33B	16,384
	DeepSeek Coder	6.7B	16,384
	CodeLLaMa	13B	16,384
	CodeLLaMa	7B	16,384

Stage ❹: Human Annotation. We engaged 13 developers to manually annotate requirements and reference dependencies for each candidate function. Given their countries of residence, all annotators obtain adequate payments.

Through discussions with annotators, we establish two criteria for requirements. *Naturalness*–ensuring the requirement reads like a natural description from the perspective of a real-world developer. *Functionality*–demanding clear descriptions of the code's purposes and input-output parameters. Each requirement undergoes a dual-annotation process, with one annotator assigned to its initial drafting and another responsible for a meticulous double-check. Trivial functions (*e.g.,* shortcut functions) and functions violating the ethical code (*e.g.,* malware) are excluded. Subsequently, the same 13 annotators review the reference code and label its reference dependencies. Finally, we retain 2,846 functions with high-quality requirements and reference dependencies.

Stage ❺: Benchmark Construction. We select candidate functions to construct based on two criteria: consistent with the data distribution of 500 real-world repositories and including as many functions as possible. Finally, we select 1,874 (73%) non-standalone functions and 706 (27%) standalone functions to construct DevEval.

4 Experiments

4.1 Studied LLMs

As shown in Table 4, we evaluate 8 popular LLMs, including two closed-source models and six open-source models. We use official interfaces or implementations to reproduce these LLMs.

4.2 Experimental Settings

Repository-level code generation takes a requirement and a repository as inputs. Typically, a repository consists of hundreds of code files and is very

Table 5: Pass@k and Recall@k of LLMs on DevEval. The bold values indicate top-1 results.

LLMs	Size	Pass@1	Pass@3	Pass@5	Pass@10	Recall@1	Recall@3	Recall@5	Recall@10
Local File (Infilling)									
gpt-4	N/A	**53.04**	**56.05**	**58.16**	**60.65**	**71.38**	**71.87**	72.90	74.12
gpt-3.5	N/A	44.50	45.48	47.56	49.85	64.46	68.15	69.22	70.78
DeepSeek Coder	33B	46.32	53.35	56.39	59.75	67.67	71.56	**73.31**	**76.13**
DeepSeek Coder	6.7B	40.82	48.13	51.44	55.11	66.27	68.33	71.09	74.36
Local File (Completion)									
gpt-4	N/A	**47.44**	**52.41**	**54.48**	**56.98**	**65.06**	**69.30**	**70.25**	71.32
gpt-3.5	N/A	40.50	45.48	47.56	49.85	60.77	64.69	66.12	67.62
DeepSeek Coder	33B	41.78	48.45	51.36	54.60	63.58	66.12	68.65	71.71
DeepSeek Coder	6.7B	36.13	43.12	46.25	50.02	61.00	64.51	66.29	68.91
StarCoder 2	15B	37.78	44.45	47.40	50.80	60.81	64.78	67.23	69.90
StarCoder 2	7B	32.82	39.29	42.28	45.77	59.71	63.02	65.84	68.83
CodeLLaMa	13B	41.94	48.83	51.92	55.66	63.33	67.68	70.16	**72.62**
CodeLLaMa	7B	39.75	46.80	49.97	53.80	60.53	65.48	67.79	70.76
Without Context									
gpt-4	N/A	**17.40**	**20.19**	**21.24**	22.55	16.85	18.53	19.56	20.98
gpt-3.5	N/A	13.98	16.38	17.51	19.01	14.90	16.69	17.06	17.95
DeepSeek Coder	33B	14.99	18.74	20.82	**23.43**	**19.03**	**21.67**	23.09	25.02
DeepSeek Coder	6.7B	12.54	17.15	19.41	22.38	17.03	18.90	20.27	22.63
StarCoder 2	15B	11.05	16.02	18.25	21.12	15.48	17.89	19.94	22.43
CodeLLaMa	13B	13.39	17.93	20.28	23.39	18.05	21.46	**23.39**	**25.94**
CodeLLaMa	7B	12.70	17.44	19.93	22.91	16.36	19.02	20.98	23.93

long. For example, the average length of 500 real-world repositories is 1.1 million tokens, surpassing the context windows of existing LLMs (*e.g.,* gpt-4: 128k tokens). Inspired by related work (Shrivastava et al., 2023), we try to extract parts of code contexts from the repository as inputs and design the following experimental settings.

❶ **Without context.** In this setting, we ignore contexts and directly generate the code based on requirements and signatures.

❷ **Local File (Completion).** The local file denotes the code file where the reference code is in. This setting simulates the scenario where developers continue to write code at the end of a file. Thus, we consider code snippets above the reference code in the local file as contexts. Then, LLMs generate code in an autoregressive manner based on requirements, signatures, and contexts.

❸ **Local File (Infilling).** Different from the Local File (Completion) setting, this setting simulates the scenario where developers infill code in the middle of a file. Thus, we use the code snippets above and below the reference code in the local file as contexts. We evaluate LLMs that support code infilling and construct input sequences using official formats.

4.3 Evaluation

We use Pass@k and Recall@k (see Section 2.3) to assess generated programs. In this paper, $k \in [1, 3, 5, 10]$. When $k = 1$, we use the greedy search and generate a single program per requirement. When $k > 1$, we use the nucleus sampling with a temperature 0.4 and sample 20 programs per requirement. We set the top-p to 0.95 and the max generation length to 500.

4.4 Main Results

The Pass@k and Recall@k of different LLMs in three experimental settings are shown in Table 5.

Without Context. gpt-4 and DeepSeek Coder achieve the highest Pass@1 and Recall@1 among all LLMs, respectively. However, all LLMs exhibit relatively low Pass@k and Recall@k values compared to their performance on previous benchmarks. For instance, gpt-4 achieves a Pass@1 score of 88.4 on HumanEval, whereas it scores 17.40 on Pass@1 in this setting. The decreases validate our motivation that existing benchmarks can not comprehensively assess the coding abilities of LLMs in real-world repositories. Furthermore, the results emphasize the importance of contexts.

Local File (Completion) and (Infilling). After

introducing the contexts within local files, the Pass@k and Recall@k of all LLMs obviously increase. For example, the Pass@1 of gpt-4 is improved by 205% and 173% in two settings, respectively.

Successful Case Analyses. We further inspect successful cases of gpt-4 and attribute the improvements to the synergy of contexts and requirements. On the one hand, the contexts provide lots of domain knowledge. For example, the local file contains essential local environments (*e.g.,* current classes, imported libraries) and a majority of dependencies (*e.g.,* intra-class and intra-file: 72% in DevEval). Recent work (Zhang et al., 2023; Ding et al., 2023) in code completion also proved the importance of contexts. On the other hand, our manually written requirements elaborate on the code's purposes and the repositories' background knowledge. Thus, the requirements help LLMs understand long contexts and locate relevant dependencies.

Error Case Analyses. Although promising, LLMs' performance in repository-level code generation is not satisfying. A manual inspection of failed cases reveals LLMs struggle with understanding contexts. Figure 4 illustrates a failed case. LLMs invoke a non-existent function - `create_connection`, even though a valid function `connect` is present in the contexts. We think two reasons cause this problem.

First, the contexts are too long. The complete repositories are lengthy, approximately 9 times the context window of the state-of-the-art LLM - gpt-4-1106. Even when partial contexts are considered, their lengths match or exceed most current LLMs' context windows. Recent work (Liu et al., 2023a) has found that LLMs often ignore relevant information in the middle of long contexts. This finding is consistent with our results. *Second, the contexts are heterogeneous.* In other words, the contexts are composed of discrete code snippets from different files rather than a continuous file. As shown in Figure 4, the programs within contexts come from multiple files, *e.g.,* `boto.regioninfo.py` and `boto.swf.layer1.py`. However, LLMs are typically trained to predict the next tokens based on the continuous contexts. The gap between training and inference objectives leads to a poor understanding of LLMs in contexts. Recent work (Shi et al., 2023) also found similar gaps in reading comprehension and question answering.

```
# Please complete the input code ...
# Here is the context:
...
# boto.regioninfo.connect
def connect(service_name, region_name, region_cls,
            connection_cls, **kw_params):
    # Create a connection class ...
    ...
# boto.swf.layer1.Layer1
class Layer1(AWSAuthConnection):
    # Low-level interface to ...
    ...
...
# Here is the input code:
# boto.swf.connect_to_region
def connect_to_region(region_name, **kw_params):
    """ Connect to a specific region in ...
    """
```

(a) Prompt

```
    # Reference Code
    # return connect('swf', region_name,
    #         connection_cls=Layer1, **kw_params)
    return create_connection(region_name, **kw_params)
```

(b) Generated Code

Figure 4: A failed case of gpt-3.5 in the Local File (Completion) setting.

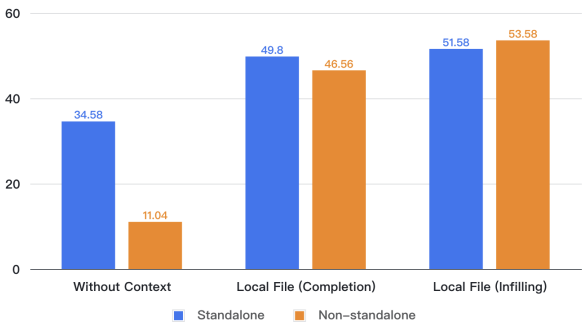

Figure 5: Pass@1 of gpt-4 on different program types.

We also obtain some interesting findings from Table 5.

❶ LLMs successfully generate some dependencies without context. Theoretically, LLMs do not see the contexts and cannot generate dependencies defined in the contexts. According to Table 5, we are surprised to find that LLMs are able to generate some dependencies without context. We manually inspect successful cases and summarize two reasons. First, LLMs can reason about some easy dependencies from requirements, *e.g.,* initialization functions of returned objects. Second, LLMs can "guess" dependencies from their functionalities. In practice, dependencies' names come from their functional descriptions, *e.g.,* `send_request()` - send a request to the server. LLMs are trained with a large code corpus and can learn the naming conventions. Thus, LLMs may successfully guess some dependencies from their functionalities.

❷ More contexts benefit code generation.

Based on Table 5, we compare the performance of an LLM (*e.g.,* gpt-4) under different settings. Obviously, the more input contexts, the better the performance of the LLM. It inspires practitioners to extend the context windows of LLMs and input more contexts.

❸ **In the without context setting, gpt family models have higher Pass@k and lower Recall@k, while other models are the opposite.** We speculate the reason is that gpt family models are instruction-tuned models and focus on performing tasks based on given instructions. With limited contexts, gpt family models are conservative and tend to generate code independently. Other LLMs are fundamental language models trained with real code files containing dependencies. They are aggressive and generate dependencies that may exist. The comparisons show the importance of instruction tuning in practical applications.

4.5 Empirical Lessons

Based on the above experiments, we summarize the empirical lessons we learned as

❶ DevEval poses new challenges, *i.e.,* repository-level code generation. The performance of existing LLMs on DevEval drops dramatically compared to their performance on previous benchmarks.

❷ LLMs benefit from code contexts in current repositories. With limited context windows, the contexts from local files can improve gpt-4 by 205% in Pass@1.

❸ Detailed and accurate requirements help LLMs know the purposes of programs and understand long contexts.

❹ LLMs struggle with understanding long and heterogeneous contexts. It causes LLMs to disregard the knowledge in contexts and even generate hallucinations (*e.g.,* non-existent functions).

5 Discussion

Results on different program types. Figure 5 shows Pass@1 of gpt-4 on different program types (*i.e.,* standalone and non-standalone). We have two observations from the results. ❶ Contexts are crucial to generating non-standalone functions. For example, adding local files improves the Pass@1 on non-standalone functions from 11.04 to 53.58. ❷ Contexts also benefit standalone functions. This is attributed to the domain knowledge within contexts, aiding LLMs in understanding requirements.

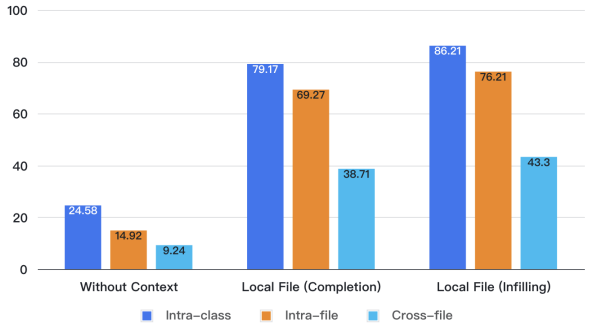

Figure 6: Recall@1 of gpt-4 on different dependency types.

❸ There is considerable room for improving LLMs on both programs. How to effectively retrieve relevant contexts is a key problem.

Results on different dependency types. Figure 6 shows the Recall@1 of gpt-4 on different dependency types (*i.e.,* intra-class, intra-file, and cross-file). The results yield two insights. ❶ Without context, LLMs can reason about some simple dependencies from requirements (*e.g.,* initialization functions of returned objects), but still exhibit low Recall@1 values across three dependency types. ❷ With contexts, LLMs exhibit an improvement in generating dependencies. Nevertheless, LLMs have yet to grapple with generating dependencies, especially cross-file dependencies. As illustrated in Figure 4, LLMs often ignore available dependencies defined in contexts.

Data leakage. Theoretically, all open-source code repositories may be included in the training data for LLMs. Consequently, there is a risk of data leakage where several repositories used to build DevEval appear in the training data. We think this risk has only a slight impact on DevEval due to three reasons. ❶ DevEval contains new data, *i.e.,* manually written requirements. These requirements are never included in the training data. ❷ Existing LLMs do not show overfitting tendencies to DevEval. Based on the release dates of 8 LLMs (see Section 4.1), we divide DevEval into two groups: unseen repositories released later than LLMs and potentially seen repositories released earlier than LLMs. The average difference of Pass@1 between the two groups is around 0.36. Compared to the average variations between LLMs (*e.g.,* 5.15 in Table 5), 0.36 is slight. ❸ DevEval is geared toward evaluating future LLMs. We release the links to our selected repositories and encourage practitioners to omit these repositories when collecting the training data

for future LLMs.

The bias of Recall@k. As stated in Section 2.3, we develop a static analysis-based parser to extract dependencies in generated programs automatically. Because Python is a dynamically typed language, certain dependencies are only determined at runtime and may elude our parser. It may lead to lower Recall@k than actual values.

To gauge the above bias, we randomly select 50 programs generated by gpt-4 and annotate dependencies with them by our parser and two human developers, respectively. Based on the human-annotated and auto-extracted dependencies, we compute two Recall@1 values. The bias of two Recall@1 values is 0.16. Compared to the average variations between LLMs (2.16 in Table 5), 0.16 is slight. Consequently, we believe that Recall@k can effectively rank different LLMs, notwithstanding its slight bias.

6 Related Work

Large Language Models for Code Generation. The rise of pre-training technology has brought new impetus to the field of code generation, both in academia and industry (Li et al., 2022; Shen et al., 2022; Nijkamp et al., 2023; Fried et al., 2023). In this context, more and more LLMs have emerged, achieving significant advancements in code generation, such as Codex (Chen et al., 2021), ChatGPT (OpenAI, 2023a), CodeLlama (Rozière et al., 2023), DeepSeek Coder (Guo et al., 2024), and StarCoder2 (Lozhkov et al., 2024).

To effectively steer LLMs in various code generation scenarios, some works focus on improving the prompt technologies by introducing specific patterns, *e.g.,* Structured Chain-of-Thought (Li et al., 2023a), Self-planning (Jiang et al., 2023), Self-debug (Chen et al., 2023), Self-collaboration (Dong et al., 2023), and AceCoder (Li et al., 2023c).

Code Generation Benchmarks. Early code generation benchmarks (Yin et al., 2018; Chen et al., 2021; Austin et al., 2021; Zan et al., 2022) evaluate code generation on relatively Python functions, such as HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021). APPS (Hendrycks et al., 2021) evaluates code generation on more difficult competition-style problems. ClassEval (Du et al., 2023) evaluates LLMs on class-level code generation and contains 100 human-crafted self-contained Python classes. Concode (Iyer et al., 2018) and CoderEval (Yu et al., 2023) further introduce non-standalone programs.

Compared to existing benchmarks, DevEval aligns with real-world code repositories (*e.g.,* the distributions of code and dependency) and contains more comprehensive annotations (*e.g.,* reference dependencies).

We have also noticed that some benchmarks have recently been proposed for repository-level tasks. CrossCodeEval (Ding et al., 2023), RepoBench (Liu et al., 2023b), and RepoEval (Zhang et al., 2023) are code completion benchmarks. They lack the necessary annotations (*e.g.,* natural language requirements) for code generation. SWE-bench (Jimenez et al., 2023) focuses on repairing repositories' issues by revising existing programs. In contrast, DevEval is collected for code generation and aims to generate new programs based on requirements for a repository. DevEval offers comprehensive annotations (*e.g.,* natural language requirements, original repositories, reference code, and reference dependencies).

7 Conclusion and Future Work

This paper proposes a new code generation benchmark named DevEval. Collected through a meticulous pipeline, DevEval aligns with real-world code repositories in multiple dimensions, *e.g.,* real code distributions, sufficient dependencies, and real-scale repositories. We evaluate 8 popular LLMs in DevEval. The results reveal the strengths and weaknesses of LLMs in real repositories. Compared to previous benchmarks, DevEval offers a more challenging and practical evaluation scenario. We hope DevEval can facilitate the applications of LLMs in practical repositories.

In the future, we will continue to update DevEval, *e.g.,* multilingual testing samples, more projects, and more test cases. Besides, we will explore how to improve the performance of LLMs in context-based code generation, *e.g.,* retrieval-augmented and tool-augmented generation.

8 Acknowledgments

This research was supported by the National Natural Science Foundation of China (Nos. 62192731, 62152730).

9 Limitations

This paper proposes a new code generation benchmark - DevEval, which aligns with real-world code

repositories. Based on DevEval, we evaluate 8 popular LLMs and analyze their strengths and shortcomings. We think that DevEval has three limitations.

❶ DevEval is a monolingual benchmark (*i.e.,* requirements in English and code in Python) and ignores other languages. In practice, LLMs require understanding requirements in different natural languages (*e.g.,* Chinese, Spanish) and generating programs in various programming languages (*e.g.,* Java, C). Thus, we plan to build a multilingual DevEval in future work.

❷ As stated in Section 5, Recall@k values in DevEval may have slight biases, *i.e.,* they may be slightly less than actual values. Because Python is a dynamically typed language, certain dependencies can only be identified at runtime and may elude our parser. To gauge the bias introduced by our parser, we manually annotate dependencies within 100 programs generated by gpt-4. Simultaneously, we employ the parser to extract dependencies in the same 50 programs. Based on the human-annotated and auto-extracted dependencies, we compute two Recall@1 values. The bias of two Recall@1 is 0.16. Compared to the average variations between LLMs (2.16 in Table 5), 0.16 is slight. Consequently, the Recall@k can effectively rank different LLMs, notwithstanding its slight bias.

❸ In our experiments, we only consider the code contexts from local files. In the future, we will explore how to utilize broader contexts (*e.g.,* imported files, sibling files).

10 Ethics Consideration

DevEval is collected from real-world code repositories. We manually check all samples in DevEval. We ensure all samples do not contain private information or offensive content. We ensure all programs in DevEval are behaving normally and exclude any malicious programs.

References

Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. 2021. Program synthesis with large language models. *CoRR*, abs/2108.07732.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating large language models trained on code. *CoRR*, abs/2107.03374.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching large language models to self-debug. *CoRR*, abs/2304.05128.

Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. 2023. Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion. *CoRR*, abs/2310.11248.

Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2023. Self-collaboration code generation via chatgpt. *CoRR*, abs/2304.07590.

Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2023. Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation. *CoRR*, abs/2308.01861.

Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. 2023. Incoder: A generative model for code infilling and synthesis. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

GitHub. 2023. Github copilot. `https://github.com/features/copilot`.

Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. Deepseek-coder: When the large language model meets programming - the rise of code intelligence. *CoRR*, abs/2401.14196.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring coding challenge competence with APPS. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*.

Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, and Luke Zettlemoyer. 2018. Mapping language to code in programmatic context. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*, pages 1643–1652. Association for Computational Linguistics.

Xue Jiang, Yihong Dong, Lecheng Wang, Qiwei Shang, and Ge Li. 2023. Self-planning code generation with large language model. *CoRR*, abs/2303.06689.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. Swe-bench: Can language models resolve real-world github issues? *CoRR*, abs/2310.06770.

Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2023a. Structured chain-of-thought prompting for code generation. *arXiv preprint arXiv:2305.06599*.

Jia Li, Yongmin Li, Ge Li, Zhi Jin, Yiyang Hao, and Xing Hu. 2023b. Skcoder: A sketch-based approach for automatic code generation. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*, pages 2124–2135. IEEE.

Jia Li, Yunfei Zhao, Li Yongmin, Ge Li, and Zhi Jin. 2023c. Acecoder: Utilizing existing code to enhance code generation. *arXiv preprint arXiv:2303.17780*.

Yujia Li, David H. Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level code generation with alphacode. *CoRR*, abs/2203.07814.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023a. Lost in the middle: How language models use long contexts. *CoRR*, abs/2307.03172.

Tianyang Liu, Canwen Xu, and Julian J. McAuley. 2023b. Repobench: Benchmarking repository-level code auto-completion systems. *CoRR*, abs/2306.03091.

Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. 2024. Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173*.

Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. Codegen: An open large language model for code with multi-turn program synthesis. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

OpenAI. 2023a. gpt-3.5-turbo. https://platform.openai.com/docs/models/gpt-3-5.

OpenAI. 2023b. GPT-4 technical report. *CoRR*, abs/2303.08774.

Pyan. 2023. Pyan. https://github.com/davidfraser/pyan.

PyPI. Pypi. https://pypi.org/.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton-Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code llama: Open foundation models for code. *CoRR*, abs/2308.12950.

Sijie Shen, Xiang Zhu, Yihong Dong, Qizhi Guo, Yankun Zhen, and Ge Li. 2022. Incorporating domain knowledge through task augmentation for front-end javascript code generation. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14-18, 2022*, pages 1533–1543. ACM.

Weijia Shi, Sewon Min, Maria Lomeli, Chunting Zhou, Margaret Li, Xi Victoria Lin, Noah A. Smith, Luke Zettlemoyer, Scott Yih, and Mike Lewis. 2023. In-context pretraining: Language modeling beyond document boundaries. *CoRR*, abs/2310.10638.

Disha Shrivastava, Hugo Larochelle, and Daniel Tarlow. 2023. Repository-level prompt generation for large language models of code. In *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pages 31693–31715. PMLR.

Pengcheng Yin, Bowen Deng, Edgar Chen, Bogdan Vasilescu, and Graham Neubig. 2018. Learning to mine aligned code and natural language pairs from stack overflow. In *Proceedings of the 15th International Conference on Mining Software Repositories, MSR 2018, Gothenburg, Sweden, May 28-29, 2018*, pages 476–486. ACM.

Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Tao Xie, and Qianxiang Wang. 2023. Codereval: A benchmark of pragmatic code generation with generative pre-trained models. *CoRR*, abs/2302.00288.

Daoguang Zan, Bei Chen, Dejian Yang, Zeqi Lin, Minsu Kim, Bei Guan, Yongji Wang, Weizhu Chen, and Jian-Guang Lou. 2022. CERT: continual pre-training on sketches for library-oriented code generation. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI 2022, Vienna, Austria, 23-29 July 2022*, pages 2369–2375. ijcai.org.

Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. Repocoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pages 2471–2484. Association for Computational Linguistics.